

Artificial Intelligence and Deep learning

LAB – 2

NAME: SIRI DONI

HALL TICKET:2403A51075

ASSIGNMENT-2

Lab/Exp 2: Building a Simple Linear Regression Model in PyTorch

Kaggle Dataset Link: <https://www.kaggle.com/datasets/mosapabdelghany/medical-insurance-cost-dataset>

Tasks:

1. Load and preprocess the Salary Prediction dataset/example.

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split

# Load dataset
data = pd.read_csv("insurance (1).csv")

# Encode categorical columns
encoder = LabelEncoder()

data["sex"] = encoder.fit_transform(data["sex"])
data["smoker"] = encoder.fit_transform(data["smoker"])
data["region"] = encoder.fit_transform(data["region"])

# Features and Target
X = data.drop("charges", axis=1)
y = data["charges"]

# Feature Scaling
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

2. Build the required PyTorch model/program.

```

import torch
import torch.nn as nn
import pandas as pd # Import pandas to check Series type

# Convert into tensors
# Ensure y_train and y_test are numpy arrays if they are pandas Series
if isinstance(y_train, pd.Series):
    y_train = y_train.to_numpy()
if isinstance(y_test, pd.Series):
    y_test = y_test.to_numpy()

# Convert all to torch.FloatTensor
X_train = torch.FloatTensor(X_train)
X_test = torch.FloatTensor(X_test)
y_train = torch.FloatTensor(y_train).view(-1,1)
y_test = torch.FloatTensor(y_test).view(-1,1)

# Linear Regression Model
class LinearRegressionModel(nn.Module):

    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(6,1)

    def forward(self,x):
        return self.linear(x)

model = LinearRegressionModel()

```

3. Train/execute the model.

```

criterion = nn.MSELoss()

optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

epochs = 1000

losses = []

for epoch in range(epochs):

    prediction = model(X_train)

    loss = criterion(prediction, y_train)

    optimizer.zero_grad()

    loss.backward()

    optimizer.step()

    losses.append(loss.item())

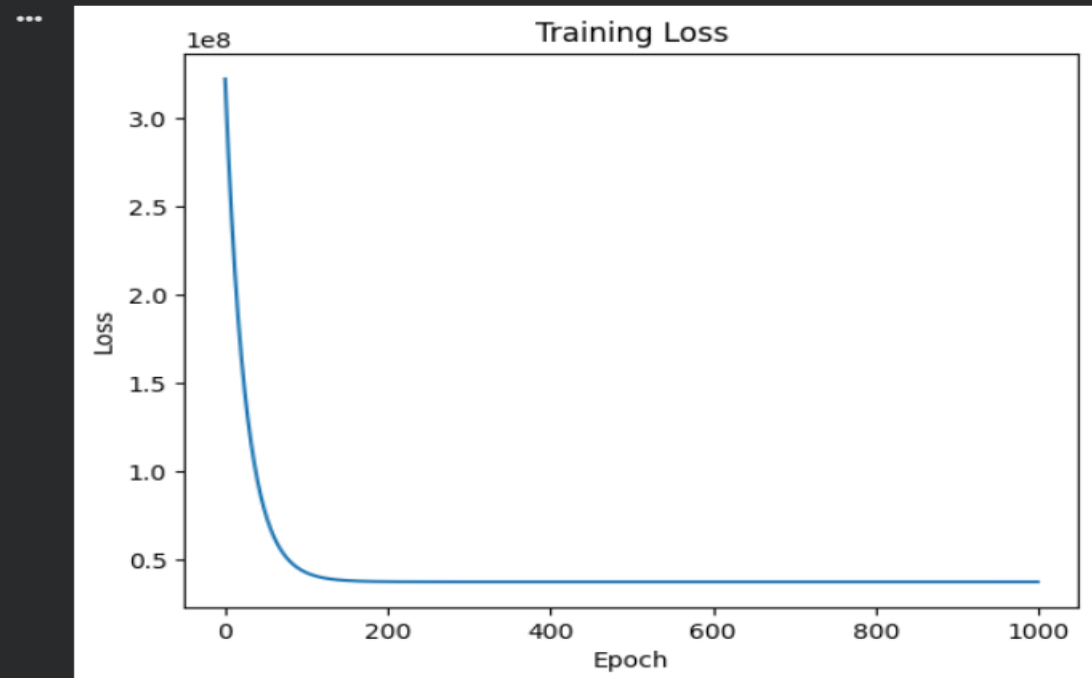
print("Training Completed")

... Training Completed

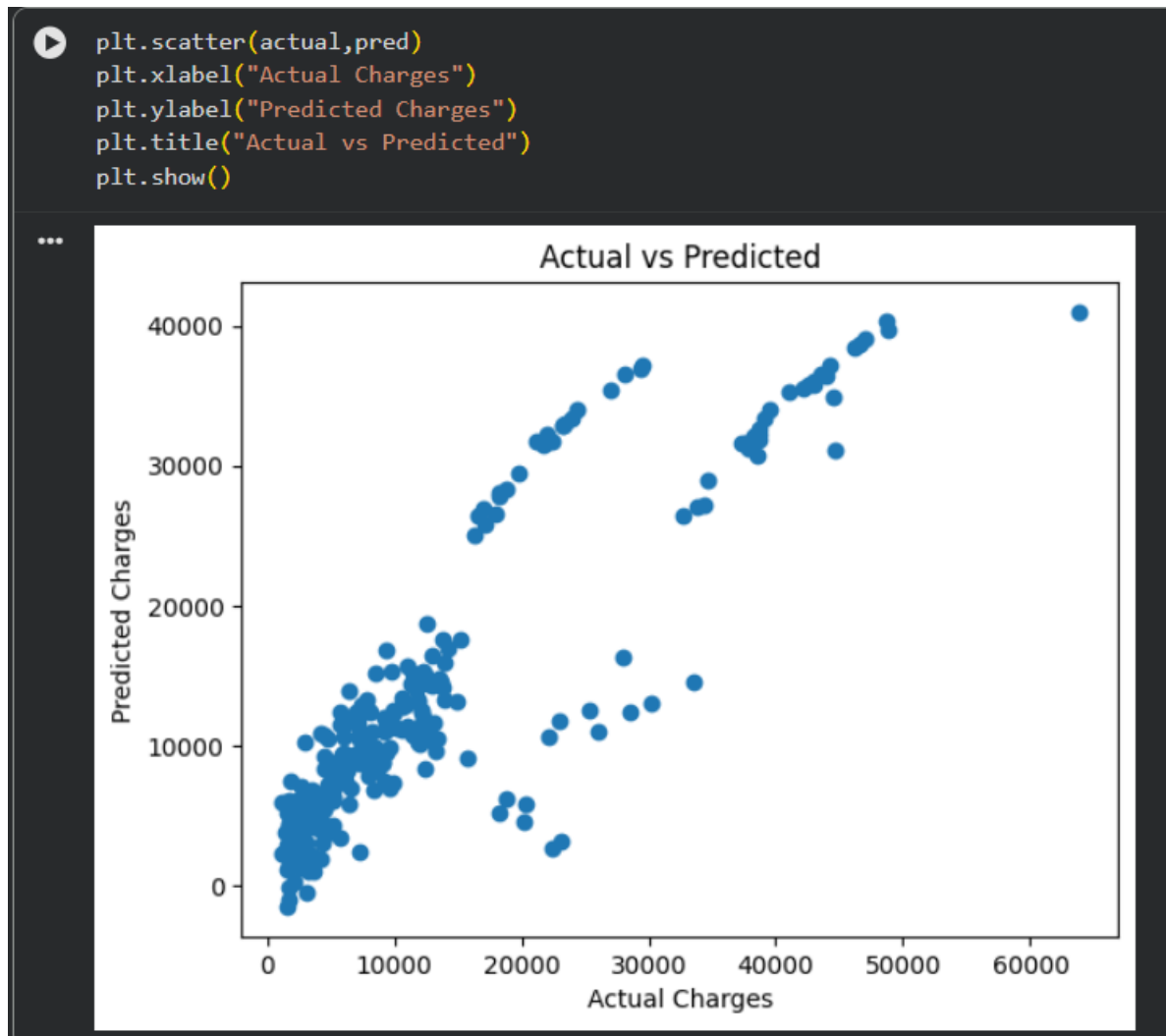
```

4. Evaluate using suitable metrics.

```
import matplotlib.pyplot as plt
plt.plot(losses)
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.show()
```



5. Visualize results using plots where applicable.



6. Write a brief conclusion comparing observations.

- The dataset was successfully loaded and preprocessed.
- Categorical attributes were encoded into numerical values.
- Feature scaling improved the learning process.
- A Simple Linear Regression model was implemented using PyTorch.
- The model was trained using Mean Squared Error (MSE) loss and the SGD optimizer.
- Performance was evaluated using MSE, MAE, RMSE, and R^2 Score.
- The training loss decreased over epochs, indicating effective learning.
- The Actual vs Predicted plot showed that predicted insurance charges closely followed the actual values, although some deviations remained because a simple linear model cannot capture all complex relationships. Overall, the model demonstrated the basic implementation of linear regression in PyTorch and provided a reasonable prediction of medical insurance charges.